

Kruskal's Algorithm - MST

MST?

- Weighted Undirected graph에서

모든 Node를 연결하면서 전체

edge weight의 합은

최소화하는 Tree

조건

- 모든 Node 연결

- No cycle

- Edge 개수 $= N - 1$

- 전체 edge weight 합이 최소

2가지 아이디어

"전체 edge를 weight가 작은 순서로 정렬하고

사이클이 생기지 않는 edge를 계속 선택"

=> 모든 edge

→ weight 기준 sorting

→ 작은 edge부터 선택

→ 사이클이 났을 때 skip

→ $N - 1$ 개까지 선택 반복

leetcode: Min Cost to Connect Points

```
n = len(points)
```

```
edges = []
```

```
for i in range(n):
```

```
    xi, yi = points[i]
```

```
    for j in range(i+1, n):
```

```
        xj, yj = points[j]
```

```
        manhattan_distance = abs(xi - xj) + abs(yi - yj)
```

```
        edges.append((manhattan_distance, i, j))
```

```
edges.sort()
```

```
uf = UnionFind(n)
```

```
res = 0
```

```
for w, u, v in edges:
```

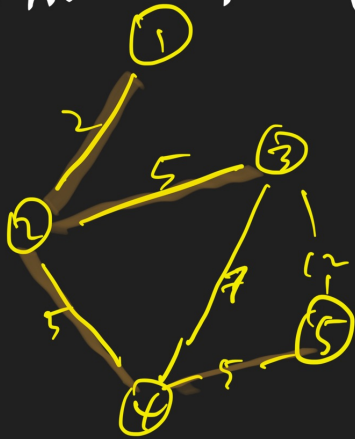
```
    if uf.union(u, v):
```

```
        res += w
```

```
return res
```

Kruskal Algorithm

- 가장 작은 비용으로 모든 노드 연결.
- 최소 비용 신장 트리 - Minimum Spanning Tree
- 여러 루트가 있고, 그 루트들을 연결한다고 생각.
- Union Find 사용



여기 연결 edge 중에
나는 최소비용이다!

MST

- 간선끼리

$$V = 5$$

$$E = 4$$

Idea : 간선을 많은 차를 더 그래프로
포함하면, 작은 것부터 작가가 되기까지
MST가 되겠네!

✱ 사이클이 생기면 안됨! → Union Find 사용

```
# Union Find  
def kruskal_algorithm (elf, graph)
```

```
    pq = []
```

```
    for u, v, w in graph:  
        pq.append((w, u, v))
```

```
    pq.sort()
```

```
    uf = UnionFind (num of node)  
    total = 0
```

```
    for w, u, v in pq:  
        if uf.union(u, v):  
            total += w
```

```
    return total
```

